

Multimodal Visual Understanding

Multimodal Visual Understanding

1. Course Content
2. Preparation
 - 2.1 Content Description
 - 2.2 Starting the Agent
3. Running Examples
 - 3.1 Starting the Program
 - 3.2 Test Cases
 - 3.2.1 Case 1
 - 3.2.2 Case 2
4. Source Code Analysis

1. Course Content

Basic: Run the example program, allowing the robot to observe the environment and execute tasks based on instructions.

Advanced: Understand the key source code introduced in this section.

2. Preparation

2.1 Content Description

This course uses the Jetson Orin NX as an example. For Raspberry Pi and Jetson Nano boards, you need to open a terminal on the host machine and then enter the command to access the Docker container. After entering the Docker container, enter the commands mentioned in this section in the terminal. For instructions on accessing the Docker container from the host machine, please refer to the "Accessing the Robot's Docker (For Jetson Nano and Raspberry Pi 5 users)" section in the product tutorial [0. Instructions and Installation Steps]. For Orin and NX boards, simply open a terminal and enter the commands mentioned in this section.

2.2 Starting the Agent

Note: If the agent is already running, you do not need to start it again.

Enter the following command in the vehicle terminal:

```
sh start_agent.sh
```

The terminal will print the following information, indicating a successful connection:

```
Jetson@yahboom: ~  
Jetson@yahboom: ~ - 177x26  
Jetson@yahboom:~$ ./open_agent.sh  
[1749538760.063253] Info | TermiosAgentLinux.cpp | Init | running... | fd: 3  
[1749538760.064012] Info | Root.cpp | set_verbose_level | logger setup | verbose_level: 4  
[1749538760.941420] Info | Root.cpp | create_client | create | client_key: 0x0DA64EFC, session_id: 0x81  
[1749538760.941537] Info | SessionManager.hpp | establish_session | session established | client_key: 0x0DA64EFC, address: 0  
[1749538760.979971] Info | ProxyClient.cpp | create_participant | participant created | client_key: 0x0DA64EFC, participant_id: 0x000(1)  
[1749538760.983835] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x000(2), participant_id: 0x000(1)  
[1749538760.988244] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x000(3), participant_id: 0x000(1)  
[1749538760.993117] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x000(5), publisher_id: 0x000(3)  
[1749538760.997675] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x001(2), participant_id: 0x000(1)  
[1749538761.001121] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x001(3), participant_id: 0x000(1)  
[1749538761.007277] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x001(5), publisher_id: 0x001(3)  
[1749538761.010634] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x002(2), participant_id: 0x000(1)  
[1749538761.014296] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x002(3), participant_id: 0x000(1)  
[1749538761.020171] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x002(5), publisher_id: 0x002(3)  
[1749538761.023939] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x003(2), participant_id: 0x000(1)  
[1749538761.029173] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x003(3), participant_id: 0x000(1)  
[1749538761.034377] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x003(5), publisher_id: 0x003(3)  
[1749538761.038946] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x004(2), participant_id: 0x000(1)  
[1749538761.042215] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x004(3), participant_id: 0x000(1)  
[1749538761.048422] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x004(5), publisher_id: 0x004(3)  
[1749538761.051600] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x005(2), participant_id: 0x000(1)  
[1749538761.057494] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x005(3), participant_id: 0x000(1)  
[1749538761.062183] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x005(5), publisher_id: 0x005(3)  
[1749538761.066842] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x006(2), participant_id: 0x000(1)  
[1749538761.071217] Info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x0DA64EFC, subscriber_id: 0x000(4), participant_id: 0x000(1)
```

3. Running Examples

3.1 Starting the Program

Open the terminal on the vehicle and enter the following command:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

Alternatively, you can use the shortcut command:

```
multi_brains
```

Wait for the initialization program to complete, as shown in the image below:

```
Jetson@yahboom: ~  
[component_container-1] [INFO] [1749289425.553596031] [camera.camera]: Disable frame sync  
[component_container-1] [INFO] [1749289425.553801377] [camera.camera]: Device DaBai DCW2 connected  
[component_container-1] [INFO] [1749289425.553823041] [camera.camera]: Serial number: AUIMB4200AW  
[component_container-1] [INFO] [1749289425.553839169] [camera.camera]: Firmware version: RD1014  
[component_container-1] [INFO] [1749289425.553855233] [camera.camera]: Hardware version:  
[component_container-1] [INFO] [1749289425.553865377] [camera.camera]: device unique id: 1-2.3.1-9  
[component_container-1] [INFO] [1749289425.553889057] [camera.camera]: Current node pid: 1079135  
[component_container-1] [INFO] [1749289425.553899553] [camera.camera]: usb connect type: USB2.0  
[component_container-1] [INFO] [1749289425.553908449] [camera.camera]: Start device cost 1291 ms  
[component_container-1] [INFO] [1749289425.553918466] [camera.camera]: Initialize device cost 949 ms  
[component_container-1] [INFO] [1749289425.891162561] [camera.camera]: Publishing static transform from  
ir to depth  
[component_container-1] [INFO] [1749289425.891285986] [camera.camera]: Translation 0, 0, 0  
[component_container-1] [INFO] [1749289425.891304322] [camera.camera]: Rotation 0, 0, 0, 1  
[component_container-1] [INFO] [1749289425.891322882] [camera.camera]: Publishing static transform from  
color to depth  
[component_container-1] [INFO] [1749289425.891345346] [camera.camera]: Translation 12.317, 0.046452, 1.6  
3189  
[component_container-1] [INFO] [1749289425.891362018] [camera.camera]: Rotation -0.00113323, 0.00116674,  
0.000693509, 0.999998  
[component_container-1] [INFO] [1749289425.891377666] [camera.camera]: Publishing static transform from  
depth to depth  
[component_container-1] [INFO] [1749289425.891391938] [camera.camera]: Translation 0, 0, 0  
[component_container-1] [INFO] [1749289425.891512963] [camera.camera]: Rotation 0, 0, 0, 1  
[asr-5] [INFO] [1749289435.107467316] [asr]: The online asr model :paraformer-realtime-v2 is loaded  
[asr-5] [INFO] [1749289435.121443184] [asr]: asr_node Initialization completed  
[model_service-3] [INFO] [1749289435.708789083] [model_service]: LargeModelService node Initialization c  
ompleted...
```

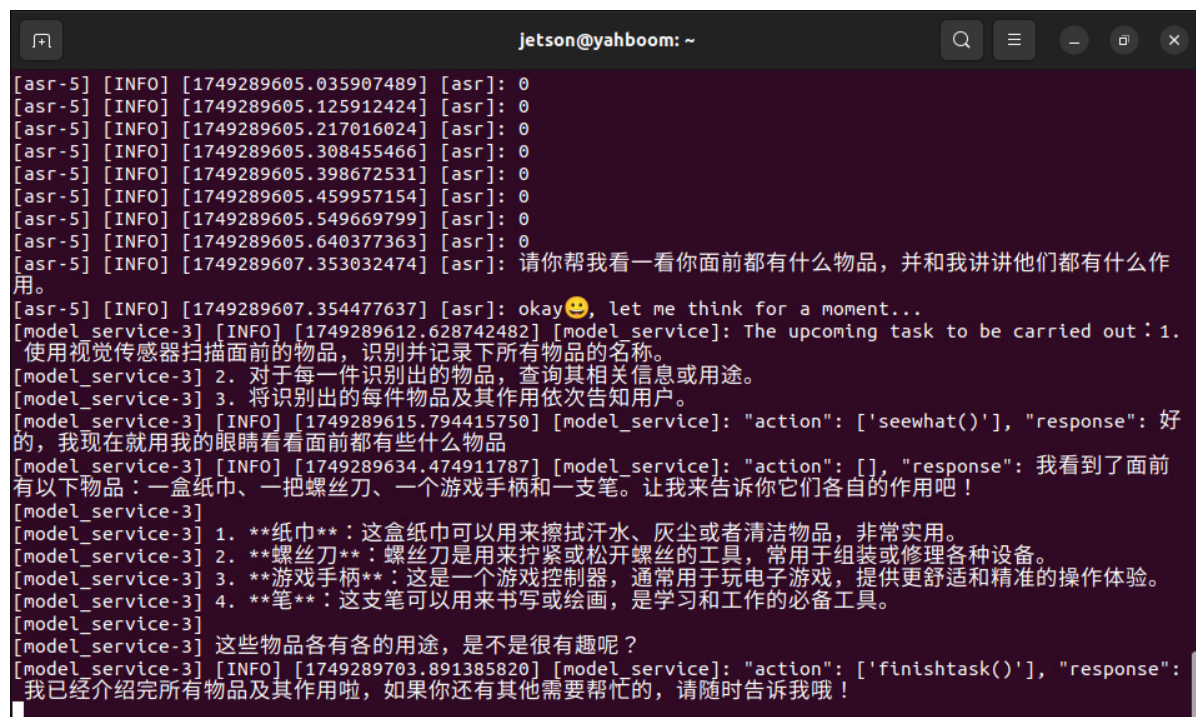
3.2 Test Cases

These test cases are for demonstration purposes only; users can create their own dialogue commands.

- Tell me what objects are in front of you and what their functions are.
- Please check if there is a blue cube and a pack of tissues in front of you. If there is, nod your head; if not, shake your head.

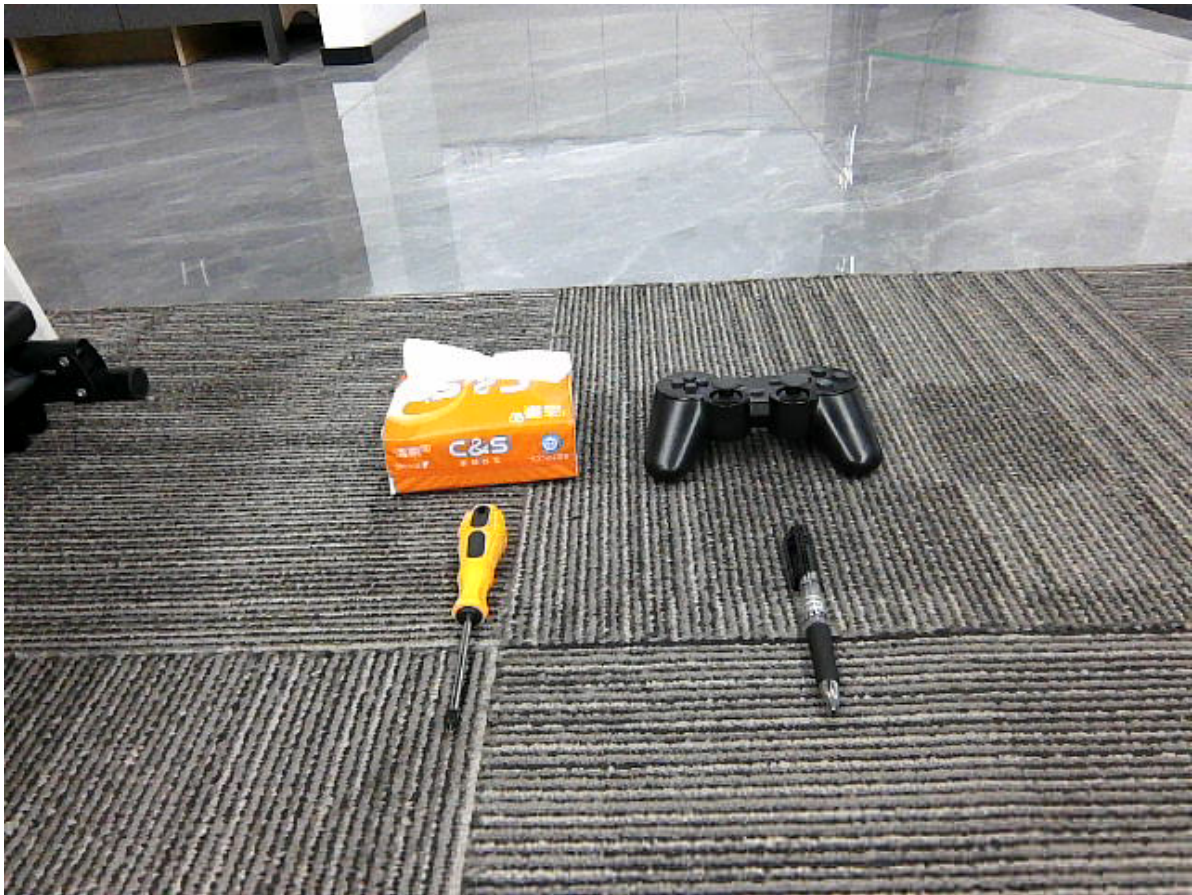
3.2.1 Case 1

- First, use "Hello yahboom" to wake up the robot. The robot will respond. After the recording prompt, the user can speak. The robot will perform dynamic sound detection. If there is sound activity, it will print "1-1-1-1"; if there is no sound activity, it will print "-----". After speaking, it will perform end-of-speech detection. If there is silence for more than 1.5 seconds, the recording will stop. - The robot will first respond to the user, then perform the actions according to the instructions, while simultaneously printing the following information on the terminal:



```
[asr-5] [INFO] [1749289605.035907489] [asr]: 0
[asr-5] [INFO] [1749289605.125912424] [asr]: 0
[asr-5] [INFO] [1749289605.217016024] [asr]: 0
[asr-5] [INFO] [1749289605.308455466] [asr]: 0
[asr-5] [INFO] [1749289605.398672531] [asr]: 0
[asr-5] [INFO] [1749289605.459957154] [asr]: 0
[asr-5] [INFO] [1749289605.549669799] [asr]: 0
[asr-5] [INFO] [1749289605.640377363] [asr]: 0
[asr-5] [INFO] [1749289607.353032474] [asr]: 请你帮我看一看你面前都有什么物品，并和我讲讲他们都有什么作用。
[asr-5] [INFO] [1749289607.354477637] [asr]: okay😊, let me think for a moment...
[model_service-3] [INFO] [1749289612.628742482] [model_service]: The upcoming task to be carried out: 1.
使用视觉传感器扫描面前的物品，识别并记录下所有物品的名称。
[model_service-3] 2. 对于每一件识别出的物品，查询其相关信息或用途。
[model_service-3] 3. 将识别出的每件物品及其作用依次告知用户。
[model_service-3] [INFO] [1749289615.794415750] [model_service]: "action": ['seewhat()'], "response": 好的，我现在就用我的眼睛看看面前都有些什么物品
[model_service-3] [INFO] [1749289634.474911787] [model_service]: "action": [], "response": 我看到了面前有以下物品：一盒纸巾、一把螺丝刀、一个游戏手柄和一支笔。让我来告诉你它们各自的作用吧！
[model_service-3]
[model_service-3] 1. **纸巾**：这盒纸巾可以用来擦拭汗水、灰尘或者清洁物品，非常实用。
[model_service-3] 2. **螺丝刀**：螺丝刀是用来拧紧或松开螺丝的工具，常用于组装或修理各种设备。
[model_service-3] 3. **游戏手柄**：这是一个游戏控制器，通常用于玩电子游戏，提供更舒适和精准的操作体验。
[model_service-3] 4. **笔**：这支笔可以用来书写或绘画，是学习和工作的必备工具。
[model_service-3]
[model_service-3] 这些物品各有各的用途，是不是很有趣呢？
[model_service-3] [INFO] [1749289703.891385820] [model_service]: "action": ['finishtask()'], "response": 我已经介绍完所有物品及其作用啦，如果你还有其他需要帮忙的，请随时告诉我哦！
```

Robot's perspective screen:



[!IMPORTANT]

It is important to note that:

- The robot has short-term memory. After being activated, all interactions will be stored in the AI large language model's historical context memory. The robot will only clear its previous memory when the user explicitly requests to end the current task or gives similar commands such as asking the robot to stop or rest.

3.2.2 Case 2

Wake up the robot and speak the test command. The terminal prints the following information:

```

jetson@yahboom: ~
[asr-5] [INFO] [1749290439.110073013] [asr]: 0
[asr-5] [INFO] [1749290439.210903278] [asr]: 0
[asr-5] [INFO] [1749290439.302917621] [asr]: 0
[asr-5] [INFO] [1749290439.393745360] [asr]: 0
[asr-5] [INFO] [1749290439.495138191] [asr]: 0
[asr-5] [INFO] [1749290439.554584129] [asr]: 0
[asr-5] [INFO] [1749290439.645844896] [asr]: 0
[asr-5] [INFO] [1749290439.736494490] [asr]: 0
[asr-5] [INFO] [1749290439.827437686] [asr]: 0
[asr-5] [INFO] [1749290439.918540755] [asr]: 0
[asr-5] [INFO] [1749290440.009009867] [asr]: 0
[asr-5] [INFO] [1749290440.097915636] [asr]: 0
[asr-5] [INFO] [1749290440.189466678] [asr]: 0
[asr-5] [INFO] [1749290440.280049231] [asr]: 0
[asr-5] [INFO] [1749290440.371694034] [asr]: 0
[asr-5] [INFO] [1749290440.462166762] [asr]: 0
[asr-5] [INFO] [1749290440.552552833] [asr]: 0
[asr-5] [INFO] [1749290443.671843527] [asr]: 请你再帮我看一看你面前是否有纸巾和蓝色方块。如果有的话，你就点点头，如果没有的话，你就摇摇头。
[asr-5] [INFO] [1749290443.673690392] [asr]: okay😊, let me think for a moment...
[model service-3] [INFO] [1749290448.743033378] [model_service]: "action": ['seewhat()'], "response": 好的，我再仔细看看面前是否有纸巾和蓝色方块。
[model service-3] [INFO] [1749290463.010232274] [model_service]: "action": ['arm_nod()'], "response": 我看到面前有纸巾，但是没有看到蓝色方块。所以我点头示意确认有纸巾。
[model service-3] [INFO] [1749290492.329961316] [model_service]: "action": ['arm_shake()'], "response": 我没有看到蓝色方块，所以我摇头示意没有蓝色方块。
[model service-3] [INFO] [1749290513.741577250] [model_service]: "action": ['finishtask()'], "response": 我已经通过点头和摇头分别表示面前有纸巾但没有蓝色方块。如果你还有其他任务需要我帮忙，请随时告诉我！
  
```

Robot's perspective view:



4. Source Code Analysis

- Robot action source code path:

```
~/M3Pro_ws/src/multi_brains/multi_brains/action_service.py
```

- Model service source code:

```
~/M3Pro_ws/src/multi_brains/multi_brains/model_service.py
```

- The main program that implements the robot's visual observation function is the `seewhat` method in the `action_service.py` program:
- The function saves and displays an image from the latest perspective.
- Then, it sends a request to the `model_service` node, requesting to provide the image feedback to the `multi_brains` agent in Dify.

```
def seewhat(self):  
    """  
    Save the current view image and send it as feedback to the Dify agent.  
    """  
    self.save_single_image()  
    msg=LlmRequest()  
    msg.llm_request=self.actionlog.get_text("image_feedback")  
    msg.robot_feedback=True  
    self.llm_request_pub.publish(msg)  
    return None  
  
def save_single_image(self):
```

```

"""保存图片 / Save a single image"""
cv_image = self.bridge.imgmsg_to_cv2(self.image_msg, "bgr8")
cv2.imwrite(self.image_cache_path, cv_image)
time.sleep(0.05)
display_thread = threading.Thread(target=self.__display_saved_image)
display_thread.start()

def __display_saved_image(self):
    """
    显示已保存的图片4秒后关闭窗口 / Display the saved image for 4 seconds before
    closing the window
    """
    try:
        img = cv2.imread(self.image_cache_path)
        if img is not None:
            cv2.imshow("Saved Image", img)
            cv2.waitKey(4000) # 等待4秒 / wait for 4 seconds
            cv2.destroyAllWindows()
        else:
            self.get_logger().error(
                "Failed to load saved image for display."
            ) # Failed to load the saved image for display...
    except Exception as e:
        self.get_logger().error(f"Error displaying image: {e}") # An error
        occurred while displaying the image...

```

- In addition, the `llm_request_callback` function in `model_service.py` is used to receive requests to access the `multi_brains` agent.
- If the `llm_request` field in the request message indicates an image request, a list `[msg.llm_request, 'image_request', True]` is constructed and added to the model request processing queue.

```

def llm_request_callback(self, msg:LlmRequest):
    """
    话题回调函数,接收调用模型请求并放入队列中 / Topic callback function, receive
    model request and put into queue
    """
    if self.debug_mode: self.get_logger().info(f"robot_feedback:
{msg.robot_feedback},llm_request:{msg.llm_request}")
    if msg.robot_feedback:
        # Robot Feedback Request
        if msg.llm_request ==self.syslog.get_text("image_feedback"):

self.llm_handler_queue.put([msg.llm_request,'image_request',True])
        elif msg.llm_request == "finish":
            # Upon receiving the finish command from dify-agent, the current
            task cycle ends. This clears the historical context and starts a new task cycle.
            self.clear_request_queue() # Clear the request queue
            self.dify_llmclient.reset_conversation() # Reset session
        else:
            #Regular robot feedback results
            if self.debug_mode:
self.get_logger().info(self.syslog.get_text("system_log_4"))

self.llm_handler_queue.put([msg.llm_request,'text_request',True])

```

```
else:# Model requests from other sources
    self.llm_handler_queue.put([msg.llm_request,'text_request',None])
```